**Seguridad web y confianza**

Diapositivas de apoyo: carátula, tres objetivos y recorrido de la información. Guion oral: leer con naturalidad, sin pronunciar los subtítulos. Ensayar a 125-145 palabras por minuto; las pausas y preguntas se agregan al tiempo de lectura.

**Apertura: lo que protegemos**

Buenas noches. Hoy vamos a conversar sobre seguridad web y, en particular, sobre ataques que afectan a los datos, a los usuarios y a las sesiones. Cuando escuchamos la palabra ataque, solemos imaginar a alguien adivinando contraseñas frente a una pantalla llena de código. Sin embargo, muchos problemas comienzan con algo mucho más cotidiano: un buscador, un comentario, un formulario o un botón que cambia nuestra información. Esas funciones son útiles porque reciben datos. El problema aparece cuando la aplicación les permite hacer algo distinto de lo que deberían hacer.

Pensemos en una plataforma de una universidad. Guarda nombres, notas y materias. Los estudiantes entran para consultar información y los docentes para modificarla. La seguridad busca que cada persona vea lo que le corresponde y realice solamente las acciones que tiene permitidas. También busca que la plataforma esté disponible cuando se necesita. Si un estudiante puede cambiar una nota ajena, hay un problema aunque conozcamos perfectamente su identidad. Estar registrado no significa tener permiso para todo. Esta diferencia será importante durante toda nuestra exposición.

**Datos, usuarios y sesiones**

Nuestro primer objetivo de protección son los datos. Aquí entran registros de estudiantes, productos, pedidos o perfiles. Una base de datos es una herramienta para almacenar y consultar esa información de manera organizada. No es insegura por existir; el riesgo depende, entre otras cosas, de cómo la aplicación le hace preguntas. Más adelante veremos la inyección SQL, que ocurre cuando una entrada logra cambiar una consulta. La palabra inyección indica precisamente esa entrada de instrucciones donde solo esperábamos un valor.

El segundo objetivo son los usuarios y su navegador. El navegador no se limita a mostrar letras: interpreta HTML, aplica estilos y ejecuta JavaScript. Por eso debemos distinguir entre mostrar algo como texto y permitir que se comporte como código. Si una aplicación confunde esas dos cosas, un comentario aparentemente normal podría ejecutar instrucciones en la página de otra persona. Ese es el centro de cross-site scripting, conocido como XSS. No significa que cualquier comentario sea peligroso; significa que el sitio debe tratarlo de la forma adecuada.

El tercer objetivo son las sesiones. Después de entrar con una cuenta, normalmente no escribimos la contraseña en cada clic. La aplicación necesita recordar que seguimos siendo la misma persona. A esa continuidad la llamamos sesión. Podemos compararla con una credencial temporal de acceso a un evento. Presentamos una identificación al ingresar y después utilizamos una pulsera durante la visita. La pulsera facilita el acceso, pero también debe protegerse. En la web, las cookies pueden transportar información que participa en ese reconocimiento.

Una cookie es un pequeño dato que el navegador guarda para un sitio y envía en las solicitudes que cumplen sus reglas. No es necesariamente una contraseña ni algo malicioso. Puede servir para una preferencia de idioma o para una sesión. El detalle importante es que el envío suele ser automático. Esa comodidad explica parte del problema de CSRF: una petición puede llegar con una credencial válida sin que la persona haya querido realizar esa acción. También explica por qué proteger la contraseña es necesario, pero no suficiente.

**El viaje de una petición**

Ahora sigamos un ejemplo sencillo. Una persona escribe su nombre en un buscador. El navegador envía una petición al servidor. El servidor es el programa que recibe esa solicitud y decide cómo responder. Puede consultar una base de datos, construir una página y devolverla. Finalmente, el navegador interpreta la respuesta. En ese recorrido hay varios lugares donde la información cambia de contexto. Un nombre escrito en un formulario no debería convertirse por accidente en una orden para la base de datos ni en una instrucción para el navegador.

A esos puntos de paso los llamamos fronteras de confianza. El término suena técnico, pero la idea es cotidiana: recibir un paquete no nos obliga a confiar en todo lo que contiene. Una aplicación debe comprobar qué espera recibir y cómo va a utilizarlo. Tampoco basta con revisar la entrada una sola vez. Un mensaje almacenado ayer sigue siendo información aportada por alguien cuando se muestra hoy. Guardar un dato no lo convierte automáticamente en seguro. Esta observación conecta los ataques a datos con los ataques a usuarios.

Conviene distinguir autenticación y autorización. Autenticar responde quién eres; autorizar responde qué puedes hacer. En una biblioteca, identificarnos con un carnet no nos permite llevarnos cualquier equipo del edificio. Del mismo modo, iniciar sesión no debería permitir consultar cualquier registro cambiando un número en una dirección. Aunque hoy nos concentraremos en SQLi, XSS y sesiones, esta distinción evita una

confusión frecuente: un sistema puede tener un buen inicio de sesión y seguir permitiendo acciones indebidas si no revisa los permisos de cada operación.

**Cómo interpretaremos los ejemplos**

Durante la exposición seguiremos la misma secuencia para cada ataque: qué significa, cómo funciona, qué puede provocar y cómo prevenirlo. El laboratorio se hará con Flask, una herramienta de Python para crear aplicaciones web. Usaremos datos ficticios y direcciones locales. La intención es observar una diferencia de comportamiento, no practicar sobre servicios de otras personas. Cuando mostremos una versión vulnerable y otra corregida, prestemos atención a cuál es la frontera que se protege. Entender esa causa resulta más útil que memorizar una cadena de prueba.

Para cerrar esta introducción, quedémonos con una idea: la seguridad web consiste en conservar el significado de lo que recibimos y comprobar cada permiso. Un nombre debe seguir siendo un nombre. Un comentario debe presentarse de manera controlada. Una sesión debe servir a la persona que la inició y durante el tiempo previsto. A partir de aquí veremos cómo una confusión entre datos e instrucciones puede afectar a una base de datos. Mi compañero continuará con la inyección SQL y con una forma concreta de evitar esa confusión.

**Apoyo para comprender y ensayar**

Servidor: programa que responde solicitudes. Base de datos: almacén organizado. Navegador: programa que interpreta la página. Petición: mensaje que solicita información o una acción. No leer este glosario como parte del monólogo; utilizarlo si aparecen dudas. Hacer una pausa después del ejemplo de la universidad y otra tras la pregunta sobre HTTPS. La pregunta puede responderla el público, pero el guion no depende de que alguien intervenga.

**Diccionario de términos**

Servidor: Programa que recibe solicitudes y envía respuestas.

Navegador: Aplicación que muestra páginas e interpreta HTML y JavaScript.

Petición HTTP: Mensaje con el que el navegador pide un recurso o solicita una acción.

Base de datos: Almacén organizado que permite consultar y modificar registros.

Autenticación: Comprobación de la identidad de una persona.

Autorización: Decisión sobre qué puede ver o hacer una identidad.

Cookie: Dato guardado por el navegador y enviado al sitio bajo determinadas reglas.

Frontera de confianza: Punto donde cambia quién controla un dato y se necesitan comprobaciones.